# SkyLink: A Concurrency-Safe Airport Seat Reservation System Built on PostgreSQL Constraints, Row-Level Security and Real-Time Replication

Amulyapreet Kaur
*Registration No. 24BAI0056*
*B.Tech CSE (Artificial Intelligence and Machine Learning)*
*Vellore Institute of Technology*
India
amulyapreet.kaur2024@vitstudent.ac.in

***Abstract*—Seat reservation is a classic test of a database system: many people try to claim the same scarce resource at the same moment, the result must never be lost or duplicated, and personal data must stay private. This paper presents SkyLink, an airport management and seat reservation system for a fictional domestic airline operating five Indian airports, 22 aircraft and 2,620 physical seats. The schema is normalised to the third normal form, and an `aircraft_model` relation is introduced specifically to remove a transitive dependency. Double booking is made impossible by a *partial unique index* on confirmed bookings rather than by application code, two `BEFORE` triggers enforce cross-table rules, and all writes go through `SECURITY DEFINER` functions. Row Level Security, column-level privileges and Supabase Auth separate anonymous visitors, signed-in passengers and administrators. A plain HTML/CSS/JavaScript front end shows a live seat map through logical replication, a personal "My bookings" page and an administrator dashboard that runs eight analytical SQL reports in a single call. In testing on PostgreSQL 16, 22 functional and security tests behaved as specified, and in five rounds of 50 clients racing for the same seat exactly one booking succeeded each time, while 50 clients booking 50 different seats all succeeded. Booking took 0.41 ms on average on the server. The results show that pushing integrity and access rules into the database yields a small, verifiable system whose guarantees do not depend on the correctness of the client.**

**Live demo:** https://itz-haachi.github.io/skylink-airways/

***Index Terms*—relational database, normalisation, concurrency control, partial unique index, row level security, PostgreSQL, Supabase, airline reservation, ACID transactions**

## I. Introduction

An airline reservation system is one of the oldest and most demanding database applications. Its core operation, *give this seat on this flight to this passenger*, looks trivial, but it has to stay correct when thousands of users act at once, when a user abandons a booking halfway, and when a malicious user edits the web page in their browser. A system that checks availability in the user interface and then inserts a booking in a second step has a race condition: two users can both see seat 12C as free and both claim it.

This project, SkyLink Airways, was built as a database systems course project with three goals:

1) **Correctness by construction.** No sequence of requests, concurrent or not, may produce two confirmed bookings for one seat on one flight, a booking for a seat that is not on the aircraft flying the flight, or a flight that lands before it departs.
2) **Privacy.** Anyone may see which seats are taken, but nobody except the passenger and the airline may see who took them.
3) **Live availability.** A seat booked in one browser should turn grey in every other open browser without a page refresh.

The central design decision is to let the database enforce every rule that matters. The web pages are deliberately thin: they draw the interface and call database functions, and they are assumed to be untrusted.

The main contributions of this work are:

- a 3NF relational schema for airports, aircraft types, aircraft, seats, flights, passengers and bookings, with the normalisation decisions and one deliberate denormalisation documented (Section III);
- a double-booking guard built from a partial unique index, which keeps the full history of cancelled bookings while allowing a cancelled seat to be sold again (Section IV-C);
- a layered security model combining database roles, Row Level Security (RLS), column-level `GRANT`s and `SECURITY DEFINER` functions, extended with Supabase Auth for passenger accounts and a table-driven administrator role (Section IV-G);
- a live seat map and an administrator dashboard that turn eight analytical queries into a visual report (Sections IV-H and IV-I);
- an evaluation with 22 functional and security tests, four

concurrency experiments and latency measurements (Section V).

The rest of the paper is organised as follows. Section II reviews the relevant database concepts. Section III presents the requirements, the ER model and normalisation. Section IV describes the implementation, Section V the testing and results, and Section VI discusses limitations. Section VII concludes.

The system is deployed and can be tried at https://itz-haachi.github.io/skylink-airways/. A demonstration administrator account is listed in Table IV.

## II. Background and Related Concepts

### A. The relational model and normalisation

The relational model represents data as relations over attribute domains and manipulates them with set-based operators [1]. Normal forms remove redundancy that would otherwise cause update, insertion and deletion anomalies [2]–[4]. A relation is in third normal form (3NF) when every non-prime attribute depends on every candidate key fully and non-transitively. In practice this is summarised as "the key, the whole key and nothing but the key".

### B. Transactions and isolation

A transaction is a unit of work with the ACID properties: atomicity, consistency, isolation and durability [5], [6]. Isolation levels trade anomalies for concurrency [7]. PostgreSQL's default level, READ COMMITTED, never shows a transaction the uncommitted changes of another (no dirty reads), but two transactions can still both *read* that a seat is free. The classic remedies are pessimistic locking, optimistic checks at commit time, and serializable isolation, which PostgreSQL implements as Serializable Snapshot Isolation [8]. A fourth option, used here, is to express the invariant as a unique constraint. When a second transaction inserts a key that an uncommitted transaction has already inserted, PostgreSQL makes it wait for that transaction to finish. It then either fails with a unique violation or proceeds [11]. The constraint becomes a lock on exactly the resource being contested [9].

### C. Partial indexes

A partial index covers only the rows that satisfy a `WHERE` predicate [12]. A *unique* partial index therefore enforces uniqueness on a subset of rows. This is exactly what a reservation system with soft-deleted (cancelled) bookings needs.

### D. Row Level Security and definer-rights functions

SQL privileges (`GRANT`/`REVOKE`) control which tables and columns a role may use. Row Level Security adds per-row predicates, called policies, that are appended to every query a role runs [13]. A function declared `SECURITY DEFINER` runs with the privileges of its owner. This makes it a controlled "window" through which a low-privilege role can perform one specific operation on data it cannot otherwise touch, provided the function pins its `search_path` [14].

TABLE I
FUNCTIONAL (F) AND NON-FUNCTIONAL (N) REQUIREMENTS

| ID | Requirement |
|---|---|
| F1 | Store airports, aircraft types, aircraft and every physical seat. |
| F2 | Schedule flights between two different airports, on a specific aircraft. |
| F3 | Show every flight with the number of seats still available. |
| F4 | Show a seat map per flight; let a passenger book one free seat and receive a PNR. |
| F5 | Let a passenger look up and cancel a booking with PNR and email. |
| F6 | Passenger accounts: sign up, sign in, and a list of one's own bookings. |
| F7 | An administrator view with revenue, load factor and cancellation reports. |
| N1 | No two confirmed bookings for the same seat on the same flight, under any concurrency. |
| N2 | Visitors may see seat status but never passenger identities. |
| N3 | All rules hold even if the browser code is modified. |
| N4 | Seat maps update in every open browser within seconds of a change. |
| N5 | Cancelled bookings are kept for history and reporting. |

### E. Backend-as-a-service

Supabase hosts a PostgreSQL database and puts three services in front of it: an automatically generated REST interface (PostgREST), an authentication service issuing JSON Web Tokens (JWTs), and a Realtime service that streams row changes from PostgreSQL's logical replication [15]–[18]. Each HTTP request is executed under the database role named in the token (`anon` or `authenticated`), so ordinary SQL privileges and RLS policies decide what a browser can do. This design moves the security boundary from a hand-written API server into the database itself [10].

## III. Requirements and Database Design

### A. Requirements

Table I lists the requirements the system was built against. They were gathered from the project brief: five airports, four to five aircraft each, seat availability, live updates and no double booking.

### B. Entity-relationship model

Fig. 1 shows the final schema. The core entities are `airport`, `aircraft_model`, `aircraft`, `seat`, `flight`, `passenger` and `booking`. Two relations were added in the final phase: `admin_user`, and a link from `passenger` to Supabase's own `auth.users` table.

Key relationships:

- An `aircraft_model` has many `aircraft`. Each aircraft has a *base* airport.
- An `aircraft` has many `seats`. A seat number such as 12C exists on every A320, so seat identity is the composite (`aircraft_id`, `seat_number`).
- A `flight` references `airport` *twice*, in the roles *origin* and *destination*, and references the aircraft that operates it.
- A `booking` is the associative entity between `passenger`, `flight` and `seat`.

TABLE II
CANDIDATE KEYS AND FUNCTIONAL DEPENDENCIES

| Relation | Candidate keys; notable dependencies |
|---|---|
| airport | {airport_id}, {iata_code} |
| aircraft_model | {model_id}, {model_name} |
| aircraft | {aircraft_id}, {registration} |
| seat | {seat_id}, {aircraft_id, seat_number}; seat_number → seat_row, seat_letter (see text) |
| flight | {flight_id}, {flight_number, departure_time} |
| passenger | {passenger_id}, {email}, {user_id} (nullable) |
| booking | {booking_id}, {pnr} |
| admin_user | {user_id} |

- An account in `auth.users` corresponds to at most one passenger (a 1:0..1 relationship enforced by a `UNIQUE` foreign key) and at most one `admin_user` row.

Every relation uses a *surrogate* primary key (an integer, or a UUID for accounts). Natural identifiers (IATA code, aircraft registration, model name, email, PNR) are declared `UNIQUE NOT NULL`, so they remain candidate keys. A surrogate key is small and fast to join, and it never changes when a real-world code does.

### C. Normalisation

*1) First normal form:* Every attribute is atomic. For example, an airport does not store a list of its aircraft; each aircraft row stores one `base_airport_id`.

*2) Removing a transitive dependency (3NF):* The first draft stored the model details on each aircraft:

*aircraft*(aircraft_id, registration, model_name, total_seats, seats_per_row).

This contains the transitive dependency *aircraft_id* → *model_name* → *total_seats*. The design repeated "A320, 180, 6" on seven rows and suffered all three anomalies:

- an update anomaly: refitting the A320 meant changing seven rows;
- an insertion anomaly: a model could not be recorded before an aircraft of that model was owned;
- a deletion anomaly: retiring the last ATR would erase the fact that an ATR 72 has 72 seats.

The relation was decomposed into

*aircraft_model*(model_id, manufacturer, model_name, total_seats, seats_per_row),
*aircraft*(aircraft_id, registration, model_id, base_airport_id).

The decomposition is lossless, because the common attribute `model_id` is a key of `aircraft_model`. It also preserves the dependencies.

*3) Functional dependencies of the final schema:* Table II lists the candidate keys and non-trivial dependencies. Every determinant except one is a candidate key, so the schema is in Boyce–Codd normal form apart from the exception discussed next.

*4) A deliberate denormalisation:* In `seat`, the seat number (12C) alone determines `seat_row` (12) and `seat_letter` (C). Because `seat_number` is only part of the candidate key (`aircraft_id`, `seat_number`), this is strictly a partial dependency, a 2NF violation. The two columns were kept on purpose. They are needed to draw the seat map and to sort seats numerically (12C after 9C), all three values are generated together and are never edited, and so no anomaly can arise in practice. Similarly, `total_seats` in `aircraft_model` is derivable as rows × `seats_per_row`. It is stored for readability, at the risk of drifting from the generated seats, and Section V verifies that it does not.

## IV. IMPLEMENTATION

The database is implemented as 17 SQL scripts executed in order (Table X in the Appendix), on PostgreSQL hosted by Supabase. The web client is six HTML pages sharing one stylesheet and one JavaScript module. Fig. 2 shows the architecture.

### A. Tables and reference data

Five airports (BLR, IXC, BOM, IXA, CCU), three models (ATR 72-600 with 72 seats in four-abreast rows, Embraer E190 with 100 in four-abreast rows, Airbus A320 with 180 in six-abreast rows) and 22 aircraft are inserted as reference data. Larger airports receive larger aircraft. Data types are chosen to be as small as correct: `CHAR(3)` for IATA codes, `SMALLINT` for model attributes, `NUMERIC(8,2)` for money (never floating point), and identity columns (`GENERATED ALWAYS AS IDENTITY`) for rows created at run time.

### B. Generating 2,620 seats with one statement

Instead of thousands of `INSERT`s, every seat of every aircraft is generated from the model's geometry (Listing 1). Two `generate_series` calls produce candidate rows 1–30 and letters 1–6; the `WHERE` clause keeps only those that exist on the model. The first two rows are business class.

Listing 1. Seat generation (02_seats.sql)

```
INSERT INTO seat (aircraft_id, seat_number,
                  seat_row, seat_letter, seat_class)
SELECT ac.aircraft_id,
       r || SUBSTR('ABCDEF', l, 1), r,
       SUBSTR('ABCDEF', l, 1),
       CASE WHEN r <= 2 THEN 'BUSINESS'
            ELSE 'ECONOMY' END
FROM aircraft ac
JOIN aircraft_model m ON m.model_id = ac.model_id
CROSS JOIN generate_series(1, 30) AS r
CROSS JOIN generate_series(1, 6)  AS l
WHERE r <= m.total_seats / m.seats_per_row
  AND l <= m.seats_per_row;
```

This yields $7 \times 180 + 10 \times 100 + 5 \times 72 = 2{,}620$ seats. The composite `UNIQUE(aircraft_id, seat_number)` guarantees that re-running the statement cannot create duplicates.

### C. Integrity: constraints and triggers

Rules that involve a single row are declared as constraints (`NOT NULL`, `UNIQUE`, foreign keys). Rules that span tables

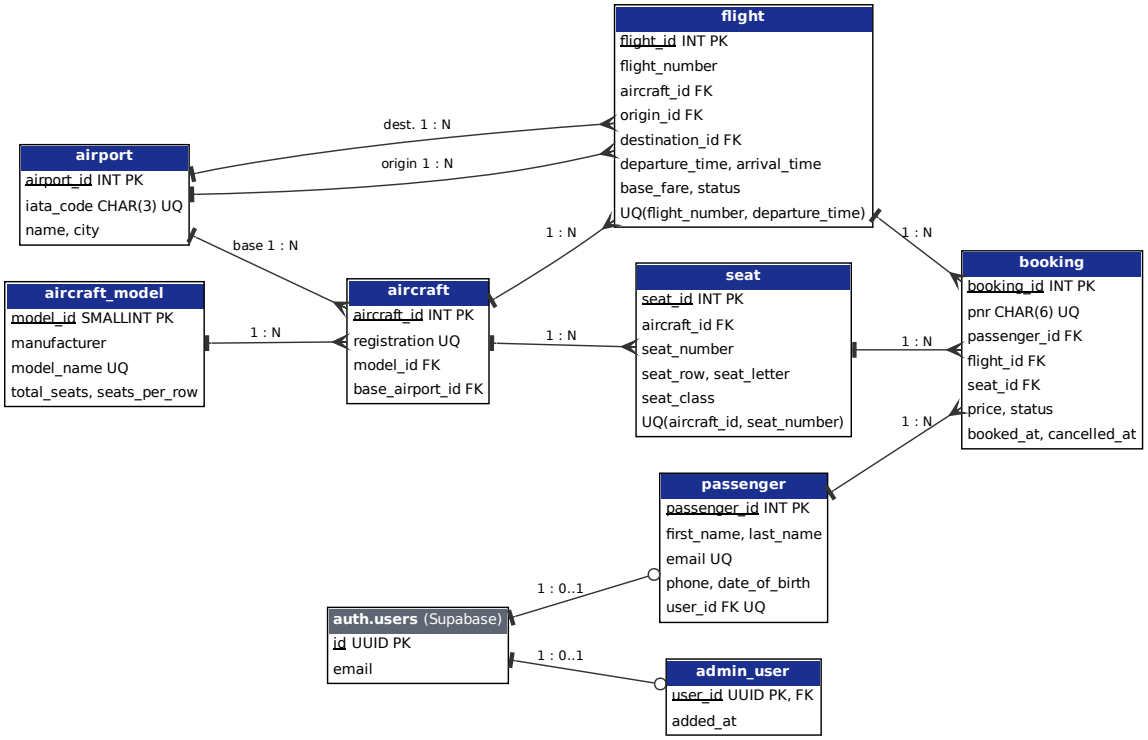



Fig. 1. Relational schema of SkyLink (crow's-foot notation). PK = primary key, FK = foreign key, UQ = unique (candidate key). `auth.users` is managed by Supabase Auth.

are enforced by two `BEFORE INSERT OR UPDATE` row triggers:

- **Trigger 1** (`check_flight_valid` on `flight`) rejects a flight whose origin equals its destination, or whose arrival is not after its departure.
- **Trigger 2** (`check_seat_on_flight` on `booking`) rejects a booking whose seat is not on the aircraft operating the flight. This relationship cannot be expressed as a simple foreign key, because it runs through `flight.aircraft_id`.

*1) The double-booking guard:* The most important rule, requirement N1, is a single index (Listing 2).

Listing 2. Partial unique indexes (05_bookings.sql)

```
CREATE UNIQUE INDEX one_booking_per_seat
  ON booking (flight_id, seat_id)
  WHERE status = 'CONFIRMED';

CREATE UNIQUE INDEX one_seat_per_passenger
  ON booking (flight_id, passenger_id)
  WHERE status = 'CONFIRMED';
```

A plain `UNIQUE(flight_id, seat_id)` would forbid ever selling a cancelled seat again, unless cancelled rows were deleted, which would destroy the history that requirement N5 asks for. The partial index covers only confirmed rows, so any number of cancelled rows may exist for one seat, but at most one confirmed row. Because the check is made by the index at insert time, it holds no matter which path the insert came from, and it serialises concurrent attempts automatically (Section IV-F).

### D. Views

Two views turn the base tables into the shapes the web pages need. Both are created `WITH (security_invoker = true)`, so they obey the permissions of the user querying them rather than those of their owner.

- `seat_map` returns every seat of every flight with a computed status: a `LEFT JOIN` to confirmed bookings, with `CASE WHEN b.seat_id IS NULL THEN 'AVAILABLE' ELSE 'BOOKED' END`.
- `flight_availability` is a view built on `seat_map`. It groups by flight and uses `COUNT(*) FILTER (WHERE ...)` to return total, booked and available seats and the load factor.

A view stores no data and is re-evaluated on every query, so seat counts are never stale.

### E. Server-side functions

Visitors never write to tables directly. Three `SECURITY DEFINER` functions form the write interface, each with `SET`

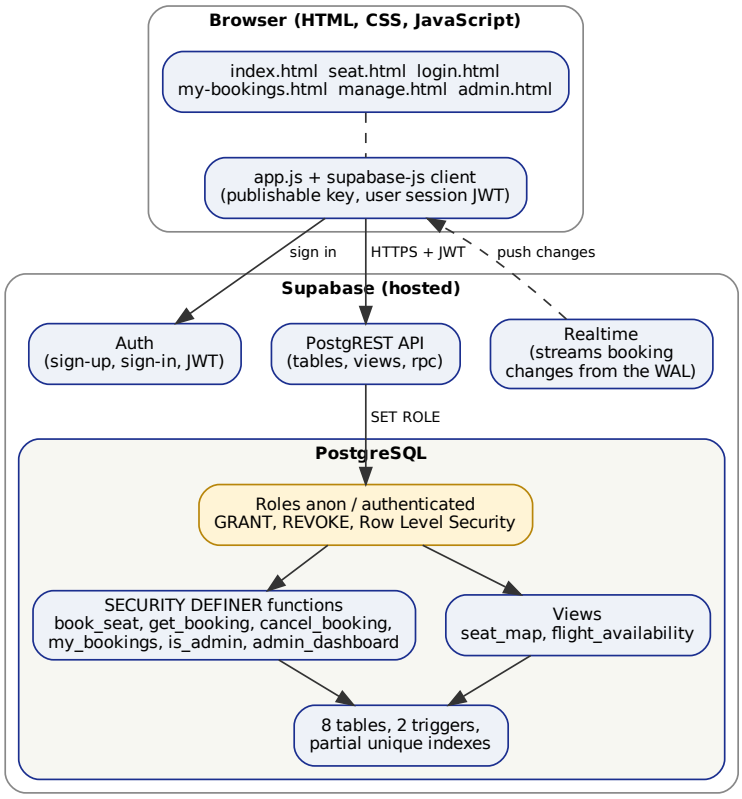



Fig. 2. System architecture. Every browser request reaches PostgreSQL under a database role taken from its JWT, so roles, RLS and grants decide what it may do.

`search_path = public` so that a caller cannot redirect it to look-alike objects.

*1) `book_seat`:* Algorithm 1 summarises the function. It validates the flight and seat, resolves the passenger, computes the price on the server (business class costs 2.5× the base fare, whatever the page displays), generates a six-character PNR from an alphabet without the ambiguous characters 0/O and 1/I, and inserts the booking. If the insert violates one of the partial unique indexes, the exception handler reads the constraint name with `GET STACKED DIAGNOSTICS` and raises a readable message. The web page shows this message unchanged.

*2) `get_booking` and `cancel_booking`:* Both require the PNR *and* the email used to book, so a guessed PNR alone reveals nothing. `get_booking` returns one joined row containing only the details shown to the passenger. `cancel_booking` locks the booking row with `SELECT ... FOR UPDATE OF b`, so two cancel requests for one PNR cannot interleave. It then refuses bookings that are already cancelled or whose flight has departed, and performs a *soft delete*: `status = 'CANCELLED'` and `cancelled_at = now()`. The partial index ignores the row from that moment, so the seat is immediately sellable again.

**Algorithm 1** `book_seat`(flight, seat, name, email, phone)

$F \leftarrow$ flight row; **if** missing, or not SCHEDULED, or departed **then** raise
$S \leftarrow$ seat with this number on $F$.aircraft; **if** missing **then** raise
**if** $uid \leftarrow$ `auth.uid()` is not null **then** ▷ signed in
  $email \leftarrow$ email in the caller's JWT ▷ ignore the form
  $P \leftarrow$ passenger linked to $uid$, else unlinked passenger with $email$ (link it)
**else** ▷ SQL editor / administrator
  $P \leftarrow$ passenger with the given $email$
**end if**
**if** $P$ is none **then** insert passenger($name$, $email$, $uid$)
$price \leftarrow F.fare \times$ (2.5 **if** $S$ is business **else** 1)
$pnr \leftarrow$ random 6 characters not already used
insert booking($pnr$, $P$, $F$, $S$, $price$) ▷ trigger 2 runs; indexes checked
**on** unique violation: map the constraint name to a message and raise
**return** $pnr$

### F. Transactions and concurrency

Each function call runs as one transaction. If any step raises an exception (for example, the booking insert fails after a new

TABLE III
ACCESS MATRIX ENFORCED BY THE DATABASE

| Object | anon | signed in | admin |
|---|---|---|---|
| airport, aircraft, seat, flight | read | read | read |
| booking: flight, seat, status | read | read | read |
| booking: pnr, passenger, price | – | – | via report |
| passenger | – | – | via report |
| admin_user | – | – | – |
| book_seat | – | call | call |
| get_booking, cancel_booking | call | call | call |
| my_bookings (own rows) | – | call | call |
| admin_dashboard | – | refused | call |

passenger row was created), the whole call is rolled back and the new passenger disappears too. Concurrency is handled at two levels:

- **Bookings.** Two transactions that try to confirm the same (flight_id, seat_id) collide in one_booking_per_seat. The second insert blocks until the first transaction ends. If the first commits, the second fails with a unique violation. If the first rolls back, the second succeeds. No explicit lock is taken, and transactions booking *different* seats never wait for each other.
- **Cancellations.** An explicit row lock (FOR UPDATE) serialises operations on one existing booking.

The script 10_transactions_demo.sql demonstrates both outcomes with two SQL editor sessions, an open transaction held with pg_sleep(30), and COMMIT or ROLLBACK. Section V reports the measured behaviour.

### G. Security model

Security is layered, and each layer is enforced by PostgreSQL (Table III).

*1) Row Level Security:* RLS is enabled on every table. A single policy, "public read" FOR SELECT USING (true), is created on the public tables for the roles anon and authenticated. No INSERT, UPDATE or DELETE policies exist, so neither role can write any row directly. passenger and admin_user have no policy at all and are invisible.

*2) Column-level privileges:* Visitors must see *which* seats are booked (to draw the map) but not *who* booked them. RLS selects rows, not columns, so this rule is expressed with column privileges:

```
REVOKE SELECT ON booking FROM anon;
GRANT  SELECT (flight_id, seat_id, status)
       ON booking TO anon;
```

The same pair is applied to authenticated, because Supabase grants that role every column by default. Without the REVOKE, a signed-in user could read every PNR.

*3) Passenger accounts:* Supabase Auth issues a JWT on sign-in, and from then on requests run as authenticated. Script 14 extends every existing policy and function grant to this role with ALTER POLICY ... TO anon, authenticated. Script 15 adds passenger.user_id UUID UNIQUE REFERENCES auth.users(id) ON DELETE SET NULL, which makes the account–passenger relationship 1:0..1 and keeps a passenger's history if the account is deleted. It also revokes book_seat from anon, so booking from the web requires a login. Inside book_seat, the email is taken from the verified token (auth.jwt()->>'email'), not from the form. A signed-in user therefore cannot book under somebody else's identity. my_bookings() returns only rows where p.user_id = auth.uid().

*4) Administrators:* Administrators are rows in admin_user. is_admin() checks membership for the caller, and admin_dashboard() begins with a guard:

```
IF NOT is_admin() THEN
  RAISE EXCEPTION 'Admins only'
        USING ERRCODE = '42501';
END IF;
```

The web page hides the Admin link from other users, but this is cosmetic. Typing the address leads to a permission error raised by the database.

### H. Live seat map

ALTER PUBLICATION supabase_realtime ADD TABLE booking adds bookings to the logical replication stream [15]. Each seat map page subscribes to changes filtered by flight_id. On any insert or update it re-queries seat_map, redraws the cabin, and briefly highlights every seat whose status differs from the previous drawing. If the passenger's own selected seat is taken by someone else meanwhile, the form is withdrawn with an explanation. Realtime applies the subscriber's RLS policy and column grants, so the broadcast never contains passenger identities [18]. Fig. 3 shows the complete booking flow.

### I. Analytical reports and the admin dashboard

Eight report queries (script 13) exercise the analytical side of SQL:

1) revenue and tickets by route (joins with two roles of airport, GROUP BY, SUM);
2) the five fullest flights (from the view, ORDER BY ... LIMIT);
3) flights above the average load factor (scalar subquery);
4) revenue by departure city (LEFT JOIN, so cities with no sales appear, plus COALESCE);
5) each class's share of revenue (window function SUM(SUM(price)) OVER ());
6) cancellation rate per flight (COUNT(*) FILTER, NULLIF, HAVING);
7) repeat passengers (HAVING COUNT(*) > 1);
8) aircraft with no flights (NOT EXISTS).

Script 17 packs all eight into admin_dashboard(). It returns a single JSONB document built with jsonb_build_object and jsonb_agg(... ORDER BY ...), so the page needs one round trip. The dashboard

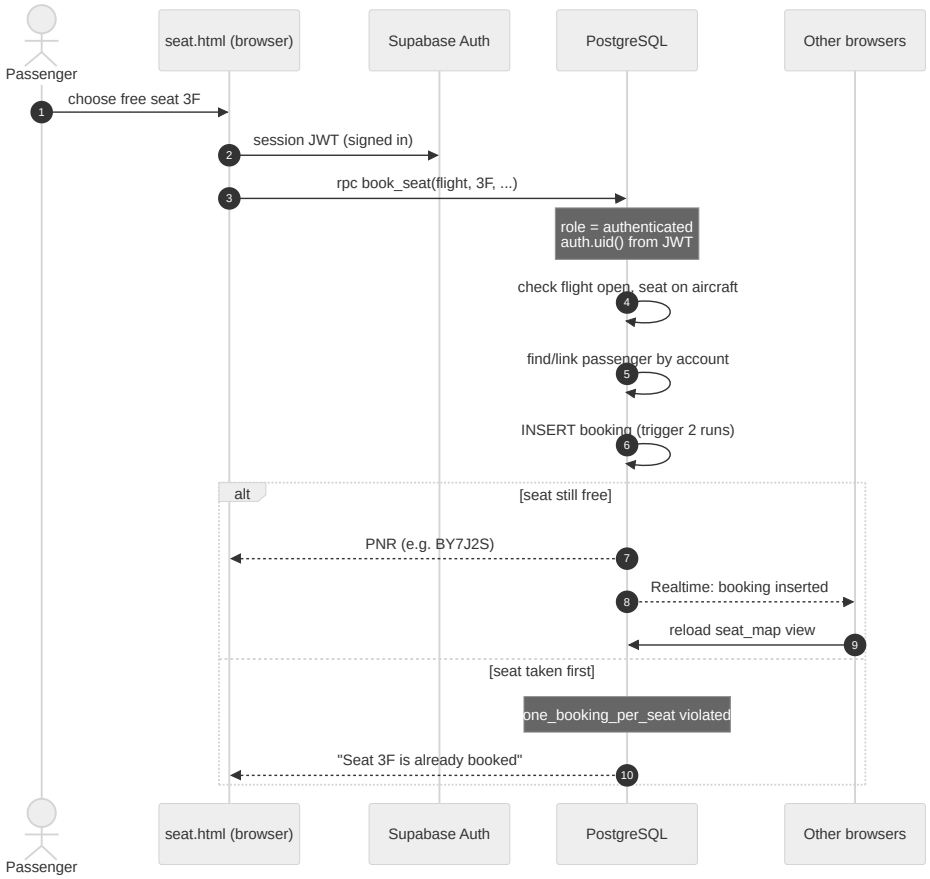



Fig. 3. Sequence of a booking. The *alt* block shows the two outcomes that the partial unique index can produce.

subscribes to booking changes as well, so revenue and load factors move while it is open.

### J. Realistic demonstration data

With only ten seed bookings, the seat maps and reports were nearly empty. Script 16 generates 300 passengers (300 distinct names from two 20-name arrays and a `LATERAL` subquery) and fills each flight to its own target between 18% and 42%, averaging 30%. It uses a chain of common table expressions:

- free seats are numbered in random order per flight with `ROW_NUMBER() OVER (PARTITION BY flight_id ORDER BY random())`;
- passengers are assigned so that no passenger receives two seats on one flight;
- a per-flight share of extra rows is inserted as cancelled.

`INSERT ... ON CONFLICT DO NOTHING` makes the script idempotent: a second run adds nothing. Seats used by the demonstration scripts are excluded, and all generated passengers share the domain `@demo.skylink.in`, so they can be removed with two `DELETE`s.

### K. Web client

The client is plain HTML, CSS and JavaScript with the `supabase-js` library and no build step. Each page reads views with `.from(...)` and calls functions with `.rpc(...)`. A shared `app.js` holds the client, formatting helpers and the header, which shows the account name, sign-out and, for administrators, the Admin link. Every database value inserted into the page passes through an escaping function, which prevents stored cross-site scripting [19]. The login page accepts a `next` parameter so that a user who chose a seat before signing in returns to the same seat. The parameter is checked against a pattern of local page names to prevent an open redirect.

The visual theme is a night sky with twinkling stars behind translucent panels. To keep text readable, a script finds the screen rectangles of all text using a `TreeWalker` over text nodes and `Range.getClientRects()`, plus every input and button. It dims any star whose box intersects one. The check is throttled with `requestAnimationFrame` and re-run on scroll, resize and content changes. Figs. 4–6 show the main screens.

### L. Deployment

Because the client is static HTML, CSS and JavaScript with no build step, it is hosted on GitHub Pages and talks to the same Supabase project used during development. Two settings in Supabase Auth make sign-up work from the public

TABLE IV
DEMONSTRATION ACCESS

| Item | Value |
|---|---|
| Website | https://itz-haachi.github.io/skylink-airways/ |
| Admin email | `admin@demo.skylink.in` |
| Admin password | `SkyLinkDemo#2026` |
| Passenger login | Create an account on the Sign in page |

address: the *Site URL* points to the GitHub Pages site, and the *Redirect URLs* allow-list contains https://itz-haachi.github.io/skylink-airways/**, so links in authentication emails can return only to this site. The browser holds only Supabase's *publishable* key. Publishing it is safe by design, because every request still runs as `anon` or `authenticated` and is limited by the policies, grants and functions of Section IV-G.

Flight times are stored relative to the day the data was loaded. Before a demonstration, the first query of `tests_and_demos.sql` shifts every flight so that the earliest departs the next day; without it, `book_seat` correctly refuses departed flights. For evaluation, an administrator account was created in the Supabase dashboard with "Auto Confirm User" and added to `admin_user` (Table IV). It can browse, book seats and open the dashboard, but, like every web user, it cannot read passenger contact details or change tables directly.

## V. TESTING AND RESULTS

### A. Environment and data set

All tests ran on PostgreSQL 16.15 (Ubuntu, 2 vCPU, 7 GB RAM). The 17 scripts were executed in order, on top of a small compatibility layer that reproduces the parts of Supabase the scripts rely on:

- the `anon` and `authenticated` roles with Supabase's default privileges;
- an `auth.users` table;
- `auth.uid()` and `auth.jwt()`, which read the request's JWT claims from a session setting exactly as on Supabase.

Requests were simulated with `SET ROLE` and those claims. Timings are server-side only; a real browser adds the network round trip to Supabase.

Table V summarises the data after one run of the generator. The seat count per model matches `total_seats` for every aircraft, confirming that the derived column has not drifted. The twelve flights offer 1,464 seats, of which 438 are confirmed (29.9%). Load factors range from 17.8% (SK202) to 42.2% (SK101).

### B. Functional and security tests

Table VI lists 22 tests, each run as a separate statement. "Refused" means the database raised the quoted error, and no data changed.

TABLE V
DATA SET AFTER SCRIPTS 01–17

| Relation | Rows | Measure | Value |
|---|---|---|---|
| airport | 5 | A320 seats (7 aircraft) | 1,260 |
| aircraft_model | 3 | E190 seats (10 aircraft) | 1,000 |
| aircraft | 22 | ATR seats (5 aircraft) | 360 |
| seat | 2,620 | Seats on scheduled flights | 1,464 |
| flight | 12 | Confirmed bookings | 438 |
| passenger | 310 | Cancelled bookings | 55 |
| booking | 493 | Average load factor | 30.0% |

### C. Concurrency experiments

Four experiments used separate database sessions running in parallel (Table VII).

- **C1 (commit).** Session A booked seat 30F on SK202 inside an open transaction held for 3 s. Session B started 1 s later. B first read the seat's status from `seat_map` and saw AVAILABLE: A's uncommitted insert was invisible, so there was no dirty read. B's booking then blocked for 2.04 s until A committed, and was refused with "Seat 30F is already booked".
- **C2 (rollback).** The same, but A rolled back. B blocked for 2.05 s and then succeeded with its own PNR. The passenger row A had created no longer existed (atomicity).
- **C3 (race).** 50 sessions were started at once, all booking the same seat under 50 different emails. This was repeated for five seats.
- **C4 (no false conflicts).** 50 sessions booked 50 *different* seats on the same flight at once.

In 250 contested booking attempts, the database produced exactly five confirmed bookings, one per seat, and every other client received the friendly message rather than an internal error. C4 shows that the guard does not serialise unrelated work.

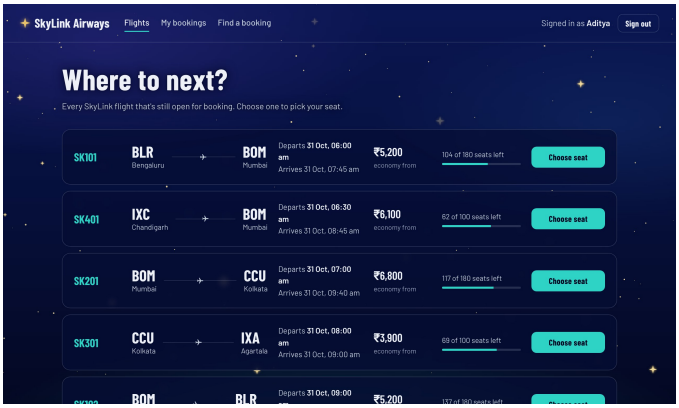
### D. Performance

Table VIII gives server-side execution times, averaged over 50 runs (100 for `book_seat`) after one warm-up run.

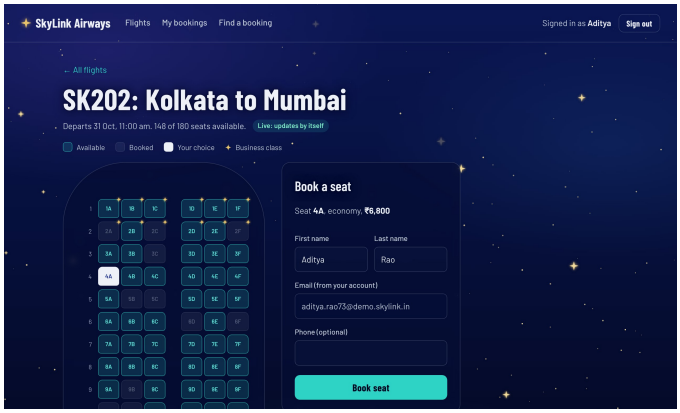
At this scale every operation finishes in a few milliseconds, so the user-visible delay is dominated by the network. `flight_availability` is the slowest public query, because it aggregates the seat map of every flight on every call; Section VI discusses how this would scale.

### E. Report results

On the generated data, confirmed revenue is INR 2,770,550. Economy contributes 81.7% of it (401 tickets) and business 18.3% (37 tickets). The busiest route by revenue is BOM→CCU (INR 479,400 from 63 tickets), and Mumbai leads the departure cities (INR 889,400). Cancellation rates range from 5.0% (SK401) to 20.0% (SK202). Sixteen of the 22 aircraft operate no scheduled flight. That is expected for a twelve-flight timetable, and it is exactly what report 8 is meant to show. These figures depend on the random generator and will differ slightly on another run.

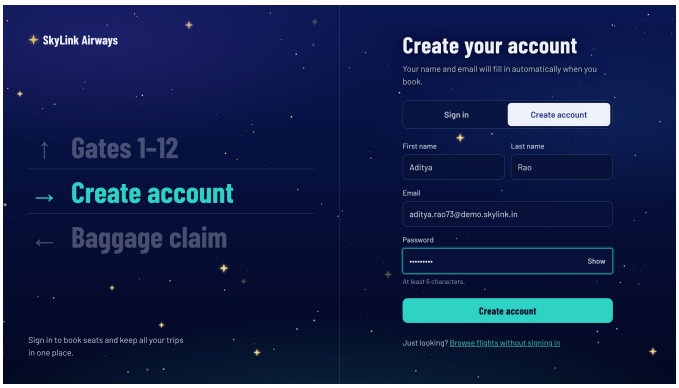



(a) Flights page reading the `flight_availability` view.

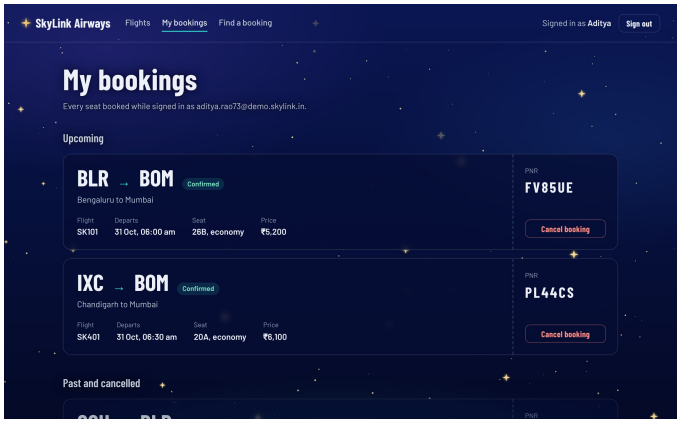



(b) Seat map of SK202 for a signed-in passenger: name and email are filled from the account, booked seats are dimmed, business class is starred.

Fig. 4. Browsing and booking.




(a) Account creation (Supabase Auth).




(b) "My bookings": rows returned by `my_bookings()` for the signed-in account only.

Fig. 5. Passenger accounts.

## VI. Discussion

### A. How ACID is achieved

Table IX maps each ACID property to the mechanism that provides it in SkyLink.

### B. Why a constraint instead of a lock

A common alternative is to lock the seat row (`SELECT ... FOR UPDATE`) or to check availability and then insert. The first adds an extra round trip and depends on every code path remembering to lock. The second is incorrect under concurrency. A unique index is declarative, applies to all paths including direct SQL (test 6), and locks only the contested key. Its main cost is that the loser learns of the conflict through an exception, which `book_seat` translates into a user-facing message.

### C. Limitations

- **Email confirmation.** For the demonstration, Supabase's "Confirm email" option was turned off. With it off, anyone could create an account using another person's address and see that person's bookings in "My bookings". A production system must keep confirmation on.
- **Shared demonstration account.** The administrator login in Table IV is public, so anyone who reads this paper can view the aggregate reports. The data is synthetic, and the account should be removed from `admin_user` after evaluation.
- **Time zones.** Times are stored as `TIMESTAMP` without a time zone. This is acceptable for a single-country airline, but international routes would need `TIMESTAMPTZ`.
- **Scope.** There are no payments, fare classes beyond two, overbooking, multi-leg itineraries or seat holds. A seat is not reserved while a user fills in the form; the database simply rejects the second person, and the live map warns them.
- **Scalability of the view.** `flight_availability` aggregates every seat on every request. With thousands of flights, it should be limited to upcoming flights, or

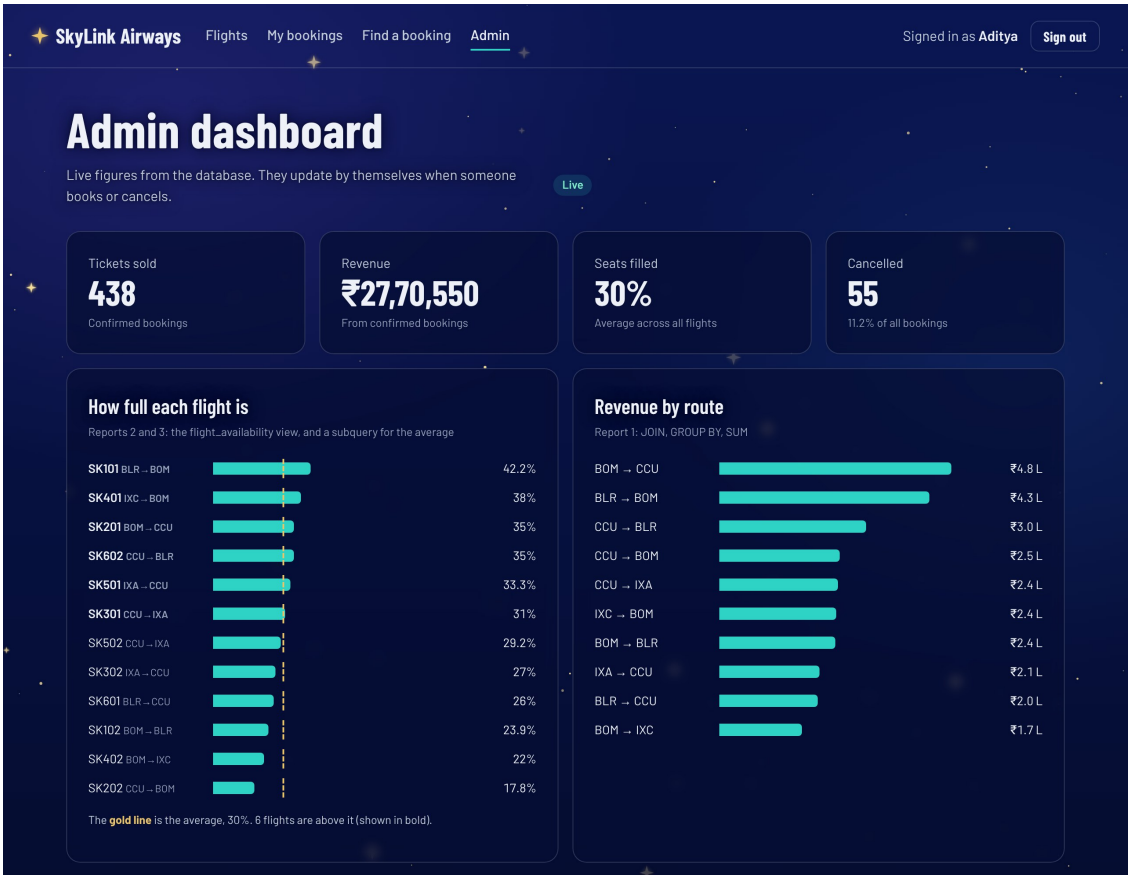



Fig. 6. Administrator dashboard (top half) for the generated data set. The gold dashed line marks the average load factor (report 3); flights above it are bold.

replaced by a counter maintained by a trigger or a materialised view refreshed on change.

- **Test environment.** The measurements come from a local PostgreSQL 16 instance with a Supabase compatibility layer, not from the hosted service.
- **Denormalised columns.** `seat_row`, `seat_letter` and `total_seats` could in principle drift from the values they are derived from. They are generated together and never edited, but a `CHECK` constraint or generated columns would make this explicit.

TABLE VI
FUNCTIONAL AND SECURITY TESTS (ALL PRODUCED THE EXPECTED RESULT)

| # | Action | Rule exercised | Observed result |
|---|---|---|---|
| 1 | Insert an airport with an existing code BLR | UNIQUE | Refused: duplicate key `airport_iata_code_key` |
| 2 | Insert an aircraft with model 9 (does not exist) | Foreign key | Refused: violates `aircraft_model_id_fkey` |
| 3 | Flight Mumbai → Mumbai | Trigger 1 | Refused: "Origin and destination cannot be the same airport" |
| 4 | Flight arriving before it departs | Trigger 1 | Refused: "Arrival time must be after departure time" |
| 5 | Book SK101 using a seat of aircraft VT-IXA1 | Trigger 2 | Refused: "This seat is not on the aircraft operating this flight" |
| 6 | Direct `INSERT` of a second booking for a taken seat | Partial unique index | Refused: `one_booking_per_seat` |
| 7 | `book_seat` on taken seat 1A of SK101 | Index + handler | Refused: "Seat 1A is already booked on flight SK101" |
| 8 | Same passenger books a second seat on SK101 | `one_seat_per_passenger` | Refused: "You already have a seat on flight SK101" |
| 9 | Book seat 45Z | Function validation | Refused: "Seat 45Z does not exist on flight SK101" |
| 10 | Cancel K7M2QX, then book its seat 1A again | Soft delete + partial index | Cancelled; new booking for 1A succeeded |
| 11 | Cancel K7M2QX a second time | Row lock + status check | Refused: "This booking is already cancelled" |
| 12 | `get_booking` with a valid PNR but wrong email | PNR + email check | 0 rows returned |
| 13 | anon: `SELECT count(*) FROM passenger` | RLS (no policy) | 0 rows visible |
| 14 | anon: `SELECT pnr FROM booking` | Column privilege | Refused: permission denied for table booking |
| 15 | anon: count confirmed bookings | Granted columns | Allowed: 438 |
| 16 | anon: direct `INSERT` into booking | RLS (no write policy) | Refused: new row violates row-level security policy |
| 17 | anon: call `book_seat` | `REVOKE EXECUTE` | Refused: permission denied for function book_seat |
| 18 | Signed in as new@x.com, form sends priya.iyer@example.com | Email from JWT | Booked; passenger email is new@x.com, linked to the account |
| 19 | `my_bookings()` for two different accounts | `auth.uid()` filter | 1 row for the booker, 0 for the other account |
| 20 | Non-admin calls `admin_dashboard()` | Admin guard | Refused: "Admins only" (SQLSTATE 42501) |
| 21 | Signed-in user reads `admin_user` | RLS + `REVOKE ALL` | Refused: permission denied |
| 22 | Admin calls `admin_dashboard()` | Admin guard | Allowed: JSON with all 8 reports |

TABLE VII
CONCURRENCY EXPERIMENTS

| Experiment | Outcome |
|---|---|
| C1 commit | B waited 2.04 s, then refused; 1 booking |
| C2 rollback | B waited 2.05 s, then succeeded; A's passenger row gone |
| C3 race, 5 rounds of 50 | Every round: 1 success, 49 "already booked", 0 other errors; exactly 1 confirmed row per seat |
| C4 50 different seats | 50 of 50 succeeded, 0.99 s wall time including client start-up |

TABLE VIII
AVERAGE SERVER-SIDE LATENCY

| Operation | Time (ms) |
|---|---|
| `book_seat` (one booking, incl. trigger and index checks) | 0.41 |
| `seat_map` for one flight (180 seats) | 0.54 |
| `get_booking` (PNR + email) | 0.60 |
| `flight_availability` (all 12 flights) | 2.64 |
| `admin_dashboard` (8 reports, one JSON) | 4.39 |

TABLE IX
ACID PROPERTIES IN SKYLINK

| Property | Mechanism and evidence |
|---|---|
| Atomicity | Each function call is one transaction; C2 left no orphan passenger. |
| Consistency | Keys, foreign keys, two triggers, two partial unique indexes; tests 1–11. |
| Isolation | Read Committed: no dirty reads (C1); conflicting inserts wait on the index entry; cancellations use `FOR UPDATE`. |
| Durability | PostgreSQL's write-ahead log; committed bookings survive restarts and are replicated to Realtime subscribers. |

## VII. Conclusion and Future Work

SkyLink shows that the hardest requirements of a reservation system can be met almost entirely inside the relational database:

- a normalised schema that keeps each fact in one place;
- a partial unique index that makes double booking impossible while keeping the full history;
- triggers for rules that span tables;
- definer-rights functions as the only write path;
- roles, RLS and column privileges for privacy.

Because these guarantees live in PostgreSQL, the browser code can be small and untrusted. The tests confirm that the rules hold whether a request comes from the web page, from direct SQL or from 50 clients at once. Live updates and the analytical dashboard come from the same database through logical replication and a single JSON-returning function.

Future work includes:

- time-limited seat holds, using a `held_until` column and an exclusion rule;
- indexes guided by `EXPLAIN ANALYZE` on larger synthetic data sets;
- time-zone-aware schedules;
- payment integration with idempotency keys;
- an audit trail implemented with triggers;
- a staff interface for scheduling flights and assigning aircraft, with an exclusion constraint preventing one aircraft from flying two overlapping flights.

## Appendix: Project Files

TABLE X
SQL scripts, in execution order

| Script | Contents |
|---|---|
| 01 | airport, aircraft_model, aircraft + reference data |
| 02 | seat; 2,620 seats via `generate_series` |
| 03 | passenger + 10 sample passengers |
| 04 | flight + trigger 1 + 12 flights |
| 05 | booking, partial unique indexes, trigger 2, samples |
| 06 | RLS, read policies, column privileges |
| 07 | views `seat_map`, `flight_availability` |
| 08 | `book_seat` |
| 09 | `get_booking`, `cancel_booking` |
| 10 | two-session COMMIT/ROLLBACK demonstration |
| 11 | Realtime publication for booking |
| 12 | audit log (designed, not deployed) |
| 13 | eight report queries |
| 14 | access for the `authenticated` role |
| 15 | `passenger.user_id`, login-aware `book_seat`, `my_bookings` |
| 16 | demonstration data (about 30% full) |
| 17 | `admin_user`, `is_admin`, `admin_dashboard` |

Live site: https://itz-haachi.github.io/skylink-airways/.
Web pages: `index.html` (flights), `seat.html` (seat map and booking), `login.html`, `my-bookings.html`, `manage.html` (find a booking by PNR), `admin.html`; shared `app.js` and `theme.css`.


## Acknowledgment

The author thanks the faculty guide for feedback during the project reviews.


## References


[1] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[2] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] T. Härder and A. Reuter, "Principles of transaction-oriented database recovery," *ACM Comput. Surv.*, vol. 15, no. 4, pp. 287–317, 1983.

[6] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.

[7] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1995, pp. 1–10.

[8] D. R. K. Ports and K. Grittner, "Serializable snapshot isolation in PostgreSQL," *Proc. VLDB Endowment*, vol. 5, no. 12, pp. 1850–1861, 2012.

[9] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Commun. ACM*, vol. 19, no. 11, pp. 624–633, 1976.

[10] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[11] PostgreSQL Global Development Group, "Index uniqueness checks," *PostgreSQL 16 Documentation*. [Online]. Available: https://www.postgresql.org/docs/16/index-unique-checks.html

[12] PostgreSQL Global Development Group, "Partial indexes," *PostgreSQL 16 Documentation*, sec. 11.8. [Online]. Available: https://www.postgresql.org/docs/16/indexes-partial.html

[13] PostgreSQL Global Development Group, "Row security policies," *PostgreSQL 16 Documentation*, sec. 5.8. [Online]. Available: https://www.postgresql.org/docs/16/ddl-rowsecurity.html

[14] PostgreSQL Global Development Group, "CREATE FUNCTION: Writing SECURITY DEFINER functions safely," *PostgreSQL 16 Documentation*. [Online]. Available: https://www.postgresql.org/docs/16/sql-createfunction.html

[15] PostgreSQL Global Development Group, "Logical replication," *PostgreSQL 16 Documentation*, ch. 31. [Online]. Available: https://www.postgresql.org/docs/16/logical-replication.html

[16] Supabase, "Row Level Security," *Supabase Docs*. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security

[17] Supabase, "Auth," *Supabase Docs*. [Online]. Available: https://supabase.com/docs/guides/auth

[18] Supabase, "Postgres Changes," *Supabase Realtime Docs*. [Online]. Available: https://supabase.com/docs/guides/realtime/postgres-changes

[19] OWASP Foundation, "Cross site scripting prevention cheat sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html